

Introduction to Computing & Programming

Subject meaning

Introduction to Computing & Programming is the bridge between computer systems and problem-solving. It typically covers computer architecture basics, data representation, algorithms, programming languages, software development, and the logic used to build correct and efficient programs.

For CSE students, the real goal is not only to learn a language syntax but to learn how to think computationally: break a problem down, design an algorithm, implement it cleanly, test it, and improve it.

Course outcomes

By the end of this subject, a student should be able to:

- Understand how a computer system processes input, storage, and output.
- Represent data using bits, bytes, numbers, and characters.
- Write programs using variables, expressions, conditions, loops, functions, and arrays/lists.
- Apply modular programming and abstraction.
- Debug and test code.
- Analyze algorithm correctness and efficiency.
- Build small applications using structured programming and, later, object-oriented thinking.

Unit 1: Computing fundamentals

This unit explains what a computer is and how it works at a basic level. Core topics usually include hardware, software, operating systems, input-output devices, memory, processors, storage, and the distinction between system software and application software.

Key ideas

- Hardware is the physical part of the computer.
- Software is the set of instructions and programs.
- The CPU executes instructions.
- Memory stores data and programs.
- Input, processing, storage, and output form the basic computing cycle.

A strong conceptual understanding of computing architecture helps you write better programs because you start thinking about how data moves through the machine.

Unit 2: Number systems and data representation

Computers represent everything using binary. This unit usually includes decimal, binary, octal, and hexadecimal number systems, along with binary arithmetic and conversion methods.

Important topics

- Bits, bytes, kilobytes, megabytes, and storage units.
- Binary addition and subtraction.
- Signed and unsigned numbers.
- Two's complement representation.
- Character encoding such as ASCII and Unicode.
- Floating-point representation at an introductory level.

This unit is important because programming errors often begin with misunderstanding how data is stored or converted inside the machine.

Unit 3: Problem solving and algorithms

This unit teaches how to convert a real problem into a structured computational solution. It typically includes algorithm design, pseudocode, flowcharts, stepwise refinement, and basic complexity ideas.

Core concepts

- Algorithm: a finite step-by-step procedure.
- Correctness: whether the algorithm solves the problem.
- Efficiency: how much time and memory it uses.
- Abstraction: focusing on essential details and ignoring irrelevant ones.
- Decomposition: splitting a problem into smaller subproblems.

Good programming starts here. If the algorithm is weak, the code will usually be weak too, even if it compiles.

Unit 4: Programming fundamentals

This unit is the core of the subject. Students usually learn a language such as C, C++, or Python, but the conceptual ideas are language-independent.

Basic syntax and structure

Programs contain:

- Keywords.
- Identifiers.
- Variables.
- Constants.
- Operators.
- Expressions.
- Statements.
- Input and output.

Data types

Common types include integers, floating-point numbers, characters, Booleans, strings, and user-defined types later in the course.

Variables and scope

Variables store values, and scope controls where a variable can be accessed. Good variable naming and clean scope usage improve readability and reduce bugs.

Input and output

Programming begins with simple input-output tasks, then expands to validation, formatting, and interactive programs.

Unit 5: Control structures

Control structures determine the order in which instructions execute. This is one of the most important parts of programming logic.

Sequence

Statements run one after another in order.

Selection

Selection includes decision-making with if, if-else, and switch/case style logic. ocw.mit.edu

BrainStack

Repetition

Repetition includes while loops, for loops, and later do-while style repetition. These are used whenever a task must be repeated many times.

Nested control

More advanced problems use nested conditions and loops. Learning to trace control flow is essential for debugging.

Unit 6: Functions and modular programming

Functions allow a program to be broken into reusable blocks. This makes code easier to test, maintain, and reuse.

Topics

- Function definition.
- Parameters and arguments.
- Return values.
- Passing by value and by reference.
- Local and global scope.
- Recursive functions.

A well-designed function should do one clear job. This is the foundation of modular programming and one of the most important habits in software development.

Unit 7: Arrays, strings, and basic data structures

As programming becomes more serious, students work with collections of data rather than single values. Typical early data structures include arrays, strings, lists, tuples, and basic record-like structures.

Core ideas

- Arrays/lists store multiple values of similar type or mixed type depending on language.
- Strings are sequences of characters.
- Indexing, slicing, traversal, and searching are essential operations.
- Basic operations include insert, delete, update, and traverse.

Data structures are central because the structure you choose affects code clarity and runtime efficiency.

Unit 8: Recursion and problem-solving patterns

Recursion is a method where a function calls itself to solve smaller instances of the same problem. It is commonly introduced after functions and loops.

Important ideas

- Base case.
- Recursive case.
- Call stack.
- Stack overflow risk.
- Recursive thinking for trees, divide-and-conquer, and backtracking.

Recursion is elegant but must be used carefully. A recursive solution is only good if it is clear, correct, and computationally reasonable.

Unit 9: Testing, debugging, and software quality

Programming is not complete until the program is tested and corrected. This unit usually includes debugging methods, testing strategies, exceptions, assertions, and good coding style.

Key topics

- Syntax errors, runtime errors, and logical errors.
- Debugging with print statements or IDE tools.
- Test cases and boundary cases.
- Assertions.
- Exception handling.
- Documentation and comments.

Writing a correct program is only half the job; making it reliable and maintainable is the other half.

Unit 10: Object-oriented programming

Many introductory courses end with object-oriented programming because it models real-world systems cleanly. Topics usually include classes, objects, methods, attributes, constructors, inheritance, polymorphism, encapsulation, and abstraction.

Why OOP matters

OOP helps you manage complexity by grouping data and behavior together. This is especially useful in larger software systems where procedural code becomes hard to manage.

BrainStack

Main pillars

- Encapsulation: hiding internal details.
- Abstraction: exposing only necessary features.
- Inheritance: reusing and extending existing classes.
- Polymorphism: same interface, different behavior.

Unit 11: Efficiency and complexity

Advanced versions of the subject often introduce algorithm efficiency, growth rates, and basic complexity notation. Students learn that some algorithms scale well and others become impractical as input size grows.

Common ideas

- Time complexity.
- Space complexity.
- Best, average, and worst case.
- Big-O notation.
- Trade-off between simplicity and performance.

This is where programming becomes computer science, not just coding.

Unit 12: Advanced computing topics overview

Some courses give a broader introduction to the discipline by touching on operating systems, networking, databases, cybersecurity, AI, and software engineering.

Common overview topics

- Operating systems and process management.
- Networking and the internet.
- Databases and data storage.
- Software development lifecycle.
- Artificial intelligence and machine learning basics.
- Security and privacy.

These topics help students see where programming fits into the larger computing ecosystem.

Programming languages and tools

Introductory courses may use C, C++, or Python. C and C++ are common when the focus is on logic, syntax discipline, memory concepts, and structured programming, while Python is often used for simpler syntax and rapid problem solving.

Students usually also learn an IDE, compiler or interpreter, debugging tools, and version-control basics later in the course path.

Course-style summary table

Area	Main idea	Why it matters
Computing fundamentals	Hardware, software, CPU, memory	Builds system understanding
Data representation	Binary, encoding, number systems	Explains how data is stored
Algorithms	Stepwise problem solving	Converts ideas into logic
Programming basics	Variables, operators, I/O	Enables coding
Control structures	Branching and loops	Creates decision logic
Functions	Modularity and reuse	Improves design
Data structures	Arrays, strings, lists	Organizes data
OOP	Classes and objects	Manages complexity
Efficiency	Big-O and scaling	Predicts performance

Detailed Unit-Wise Notes for Introduction to Computing & Programming

Unit 1: Introduction to computing

Computing is the process of using computers to represent, process, store, and communicate information. In a CSE course, this unit builds the foundation by explaining what a computer is, how it works, and why abstraction is central to all software systems.

A computer system typically consists of hardware, software, data, users, and procedures. Hardware includes the physical components such as CPU, memory, input devices, output devices, and storage, while software includes operating systems, utilities, and application programs.

Core concepts

- **Hardware:** Physical components of a computer.
- **Software:** Programs that instruct hardware.
- **Firmware:** Low-level software embedded in devices.
- **Data:** Raw facts that become information after processing.
- **Information:** Processed data with meaning.
- **Abstraction:** Hiding unnecessary complexity.

Functions of a computer

- Input.
- Processing.
- Storage.
- Output.
- Control.

Computer generations

Computer generations are often introduced to show the historical development of computing, from vacuum tubes to transistors, integrated circuits, microprocessors, and modern intelligent systems. This gives students context for why computing power and efficiency improved over time.

Importance

Understanding how a computer system is built helps programmers write better code because software performance, memory usage, and execution style all depend on the underlying machine.

Unit 2: Number systems and data representation

All computer data is represented in binary. This unit explains how numbers, characters, images, and instructions are encoded using bits and bytes.

Number systems

The common systems are:

- Decimal.
- Binary.
- Octal.
- Hexadecimal.

Conversions

Students should learn:

- Decimal to binary.
- Binary to decimal.
- Binary to octal and hexadecimal.
- Hexadecimal to binary.

Binary arithmetic

Important operations include binary addition, subtraction, multiplication, and simple overflow understanding. These operations are basic but essential for later computer architecture and programming topics.

Signed representation

Common signed integer systems include sign-magnitude, one's complement, and two's complement. Two's complement is the most widely used because it simplifies arithmetic in digital systems.

Character encoding

Computers represent text using encodings such as ASCII and Unicode. ASCII uses a smaller code set, while Unicode supports a much wider range of world languages and symbols.

Why it matters

Many programming bugs come from misunderstanding storage size, overflow, character encoding, or precision. so this unit is not just theoretical; it directly affects practical coding.

Unit 3: Problem solving and algorithmic thinking

Programming begins with problem solving. This unit teaches how to transform a real-world problem into a logical sequence of steps that a computer can execute.

What is an algorithm?

An algorithm is a finite, ordered set of instructions to solve a problem. A good algorithm must be correct, unambiguous, finite, and efficient.

Steps in problem solving

1. Understand the problem.
2. Identify inputs and outputs.
3. Break the problem into smaller parts.
4. Design the logic.
5. Write pseudocode or flowchart.
6. Implement in a programming language.
7. Test and debug.

Pseudocode

Pseudocode is a language-independent way to express logic in simple steps. It helps students think clearly before writing actual code.

Flowcharts

Flowcharts use standard symbols to show program logic visually. They are useful for tracing decisions, loops, and sequences.

Algorithm properties

- **Input:** one or more inputs.
- **Output:** at least one result.
- **Definiteness:** each step is clear.
- **Finiteness:** it terminates.
- **Effectiveness:** each step is executable.

Unit 4: Programming fundamentals

This unit introduces actual coding concepts. Although different colleges may use different languages, the fundamental ideas remain the same across C, C++, Java, and Python.

Program structure

A program generally contains:

- Input statements.
- Variable declarations.
- Expressions and assignments.
- Control structures.
- Function calls.
- Output statements.

Variables and constants

Variables store values that can change, while constants store fixed values. Good naming makes code easier to read and debug.

Data types

Common data types include:

- Integer.
- Floating-point.
- Character.
- Boolean.
- String.

Operators

Operators are used for:

- Arithmetic.
- Relational comparison.
- Logical operations.
- Assignment.
- Bitwise operations in more advanced study.

Expressions and statements

Expressions combine values and operators to produce a result. Statements are executable instructions. Understanding the difference is essential for writing valid programs.

Input and output

Input allows the program to receive data from the user; output shows results. Clean input/output handling is a basic but critical programming skill.

Unit 5: Control structures

Control structures decide the order of execution in a program. They are one of the most important parts of introductory programming.

Sequence

Sequence means statements execute in order from top to bottom.

Selection

Selection is decision-making using constructs like if, if-else, nested if, and switch/case. It is used when the program must choose among alternatives.

Iteration

Iteration means repeating instructions using loops such as for, while, or do-while. Loops are used for calculations, traversal, counting, validation, and repeated tasks.

Nested control

Complex problems often require nested if statements or nested loops. These structures must be traced carefully to avoid logical errors.

Best practice

Always write a loop with clear initialization, condition, and update steps. Many beginner bugs occur because one of these three parts is missing or wrong.

Unit 6: Functions and modular programming

Functions break a program into small reusable units. This is a key step from “writing code” to “designing software.”

Function components

A function usually has:

- Name.
- Parameters.
- Body.
- Return type or return value.

Why functions matter

- Reuse code.
- Reduce repetition.
- Improve readability.
- Simplify testing.
- Support abstraction and modularity.

Arguments and parameters

Parameters are the variables in the function definition, while arguments are the actual values passed when calling the function.

Scope

Local variables exist inside a function, while global variables exist outside. Understanding scope prevents unexpected behavior and name conflicts.

Recursion

Recursion is a function calling itself to solve smaller versions of the same problem. It requires a base case to stop and a recursive case to continue.

Unit 7: Arrays, strings, and basic data structures

This unit introduces data organization. Instead of handling one value at a time, programs often work with groups of values.

Arrays

An array stores elements of the same type in contiguous memory in many languages. Key operations are traversal, insertion, deletion, searching, and updating.

Strings

A string is a sequence of characters. Common operations include length, concatenation, comparison, indexing, slicing, and pattern checking.

Why arrays and strings matter

Many algorithmic problems in CSE begin with arrays and strings because they are easy to store, traverse, and process.

Introductory data structures

Depending on the course, students may also see:

- Lists.
- Stacks.
- Queues.
- Simple records or structures.

Unit 8: Debugging, testing, and program quality

Writing code is incomplete without testing. This unit teaches how to find and fix errors and how to improve reliability.

Types of errors

- Syntax errors: grammar violations.
- Runtime errors: occur while executing.
- Logical errors: program runs but gives wrong output.

Debugging methods

- Use print statements.
- Trace variables step by step.
- Use debugger tools.
- Isolate faulty sections.
- Check boundary conditions.

Testing

Testing includes normal cases, boundary cases, and invalid cases. Good testing ensures the program behaves correctly in different situations.

Code quality

Good code is readable, modular, well-named, documented, and easy to maintain. This is a major professional skill in computer science.

Unit 9: Object-oriented programming

Many introductory programming courses end with object-oriented programming because it is a natural way to model real systems.

Core concepts

- Class.
- Object.
- Attribute.
- Method.
- Constructor.
- Encapsulation.
- Abstraction.
- Inheritance.
- Polymorphism.

Encapsulation

Encapsulation groups data and methods into one unit and restricts direct access to internal details.

Inheritance

Inheritance allows one class to reuse and extend another class, reducing duplication and supporting hierarchical design.

Polymorphism

Polymorphism means the same interface can behave differently depending on the object type.

Importance

OOP is powerful for large systems, code reuse, and maintainable architecture.

Unit 10: Efficiency and computational thinking

Advanced versions of the course often include introductory complexity analysis. This unit teaches students how to think about performance, not just correctness.

Main ideas

- Time complexity.
- Space complexity.
- Best case, average case, worst case.
- Big-O notation.

Why it matters

Two programs can produce the same result, but one may run much faster or use much less memory. In CSE, this difference becomes important as data size grows.

Computational thinking

Computational thinking includes decomposition, pattern recognition, abstraction, and algorithm design. It is a central mindset in computer science.

Final study strategy

Start with computer fundamentals and number systems, then move into algorithms and basic programming, then functions and data structures, and finally OOP and complexity. This sequence mirrors how the subject naturally builds from machine understanding to program design.